

OS_project03_10831

2022083409 컴퓨터소프트웨어학부 김승관

Design – Copy-on-Write Fork

- kernel/proc.c – fork(), wait()
 - 자식 process를 fork()할 때 부모 process의 page table을 uvmcopy()를 사용해 자식 process의 page table로 복사한다. 또한 copyout()을 사용해 kernel space의 data를 user address space에 복사한다.
 - Copy-on-Write에서는 fork() 시에 부모 process의 page table을 자식 process가 똑같이 공유하고 있다가 data를 write 할 경우에 새로운 page를 만들고 그곳에 기존 page table을 복사한 뒤 수정을 해야한다.
 - 따라서, COW fork를 위해 uvmcopy()와 copyout()의 수정이 필요하다.
 - 또한 이에 더해 usertrap(), kalloc()/kfree()를 수정해줘야 한다.

fork()

```
// Copy user memory from parent to child.
if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
    freeproc(np);
    release(&np->lock);
    return -1;
}
```

wait()

```
if(pp->state == ZOMBIE){
    // Found one.
    pid = pp->pid;
    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
        sizeof(pp->xstate)) < 0) {
        release(&pp->lock);
        release(&wait_lock);
        return -1;
    }
    freeproc(pp);
}
```

Implementation Overview

- kernel/vm.c
 1. uvmcopy() 수정
 2. copyout() 수정
- kernel/riscv.h
 1. PTE_COW flag 추가
- kernel/trap.c
 1. usertrap() 수정
- kernel/kalloc.c
 1. kalloc() 수정
 2. kfree() 수정
 3. incref() 추가

Implementation – Copy-on-Write Fork

- kernel/vm.c

1. uvmcopy() 수정

자식 process가 생성될 때 부모 page table을 공유해야 한다.. 이때, 자식 process의 PTE_W flag를 제거하고 PTE_COW flag를 추가한다. incref()를 사용해 현재 page의 refer count를 증가시킨다.

이후 자식 process의 page table을 동일한 physical address로 mapping 해준다.

부모 page table도 flag를 갱신해준다.

- kernel/riscv.h

1. PTE_COW flag 추가

PTE 구조에 따라서 COW flag를 8번 bit로 한다.

```
#define PTE_COW (1L << 8) // COW flag
```



```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
    uint64 pa, i;
    uint flags;
    // char *mem;

    for(i = 0; i < sz; i += PGSIZE){
        if((pte = walk(old, i, 0)) == 0)
            panic("uvmcopy: pte should exist");
        if((*pte & PTE_V) == 0)
            panic("uvmcopy: page not present");
        pa = PTE2PA(*pte);
        flags = PTE_FLAGS(*pte);

        flags &= ~PTE_W;
        flags |= PTE_COW;

        incref(pa);

        if(mappages(new, i, PGSIZE, pa, flags) != 0){
            goto err;
        }

        *pte = PA2PTE(pa) | flags;
    }
    return 0;
}
```

Implementation – Copy-on-Write Fork

- kernel/vm.c
 2. copyout() 수정
PTE에 PTE_W가 없으면 원래 return -1이었으나, COW인지 확인이 필요해졌다.
PTE_COW가 있을 경우 kalloc()으로 새 page를 할당 해 복사할 준비를 한다.
memmove()로 기존 page 내용을 새 페이지로 복사한다.
새 page는 PTE_COW를 제거하고 PTE_W flag를 추가한다.
기존 COW page를 unmap한 뒤, 새로 복사된 page를 mappage()를 통해 다시 매핑한다.
원래 공유하던 page는 kfree()를 통해 해제하는데, ref count가 0일 때만 해제한다.
이후 기존 코드대로 kernel에서 user space로 데이터를 복사한다.

```
int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    uint64 n, va0, pa0;
    pte_t *pte;

    while(len > 0){
        va0 = PGROUNDDOWN(dstva);
        if(va0 >= MAXVA)
            return -1;
        pte = walk(pagetable, va0, 0);
        if(pte == 0 || (*pte & PTE_V) == 0 || (*pte & PTE_U) == 0)
            return -1;

        if((*pte & PTE_W) == 0){
            // COW 처리
            if((*pte & PTE_COW) == 0)
                return -1;

            uint64 pa = PTE2PA(*pte);
            char *mem;
            if((mem = kalloc()) == 0)
                return -1;
            memmove(mem, (char*)pa, PGSIZE);

            uint flags = PTE_FLAGS(*pte);
            flags &= ~PTE_COW;
            flags |= PTE_W;

            uvmunmap(pagetable, va0, 1, 0);

            if (mappages(pagetable, va0, PGSIZE, (uint64)mem, flags) != 0) {
                kfree(mem);
                return -1;
            }

            kfree((void*)pa);
        }
    }
}
```

Implementation – Copy-on-Write Fork

The RISC-V Instruction Set Manual:
Volume II

Privileged Architecture

Version 20250508: This document is in Ratified state.

15 Store/AMO page fault

- kernel/trap.c

1. usertrap 수정

RISC-V xv6 manual을 봤을 때 scause=15는 store page fault를 말한다. user가 write를 logical address에 PTE_W가 없는 경우 발생한다.

PTE_COW로 표시된 공유 page에 user가 write를 시도하면 이 page에는 PTE_W가 없기 때문에 왼쪽에 있는 usertrap()의 해당 조건문이 실행된다.

r_stval()로 fault를 일으킨 logical address를 얻고 page 단위 작업을 위해 page 기준으로 내림 정렬을 한다.

walk()를 통해 해당 logical address에 대한 PTE를 찾고 COW page인지 판단을 한 뒤, COW page가 맞으면 physical address를 얻는다.

새 physical page를 할당해 기존 공유 page 내용을 memmove()를 사용해 복사한다. 이때 PTE_COW를 제거하고 PTE_W를 추가한다. 기존 공유하던 PTE를 unmap하고 새로운 physical page를 기존 logical address에 mapping한다.

이후 kfree()로 원래 공유하던 page table을 해제하는데, ref count가 0이 되면 해제한다.

```
else if(r_scause() == 15){ // store page fault
    uint64 va = r_stval();
    va = PGROUNDDOWN(va);

    pte_t *pte = walk(p->pagetable, va, 0);
    if(pte && (*pte & PTE_V) && (*pte & PTE_U) && (*pte & PTE_COW)){
        uint64 pa = PTE2PA(*pte);
        char *mem = kalloc();
        if(mem == 0){
            setkilled(p);
            return;
        }

        memmove(mem, (char*)pa, PGSIZE);

        uint flags = PTE_FLAGS(*pte);
        flags &= ~PTE_COW;
        flags |= PTE_W;

        uvmunmap(p->pagetable, va, 1, 0);
        if(mappages(p->pagetable, va, PGSIZE, (uint64)mem, flags) != 0){
            kfree(mem);
            setkilled(p);
            return;
        }
        kfree((void*)pa);
    }
}
```

Implementation – Copy-on-Write Fork

- kernel/kalloc.c

1. incref() 추가

전역 변수로 ref_count를 만들고 ref_lock을 추가해 race condition을 방지했다.

physical address를 page 단위 index로 변환하기 위해 pa를 PGSIZE로 나눈 값을 index로 사용했다.

ref_count[pa/PGSIZE]++은 해당 page가 공유된 횟수가 1 증가했다는 뜻이다.

2. kalloc() 수정

kalloc()을 호출할 때 ref_count를 1 증가시킨다.

3. kfree() 수정

kfree()를 호출할 때 ref_count를 1 감소시킨다. 이때 ref_count가 0 이하이면 해당 page를 free해준다.

incref()

```
#define MAX_PAGES (PHYSTOP / PGSIZE)
int ref_count[MAX_PAGES];
struct spinlock ref_lock;

void incref(uint64 pa) {
    acquire(&ref_lock);
    ref_count[pa / PGSIZE]++;
    release(&ref_lock);
}
```

kfree()

```
void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    acquire(&ref_lock);
    int *ref = &ref_count[(uint64)pa / PGSIZE];
    (*ref)--;
    if(*ref > 0) {
        release(&ref_lock);
        return;
    }
    release(&ref_lock);

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

kalloc()

```
void *
kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r)
        kmem.freelist = r->next;
    release(&kmem.lock);

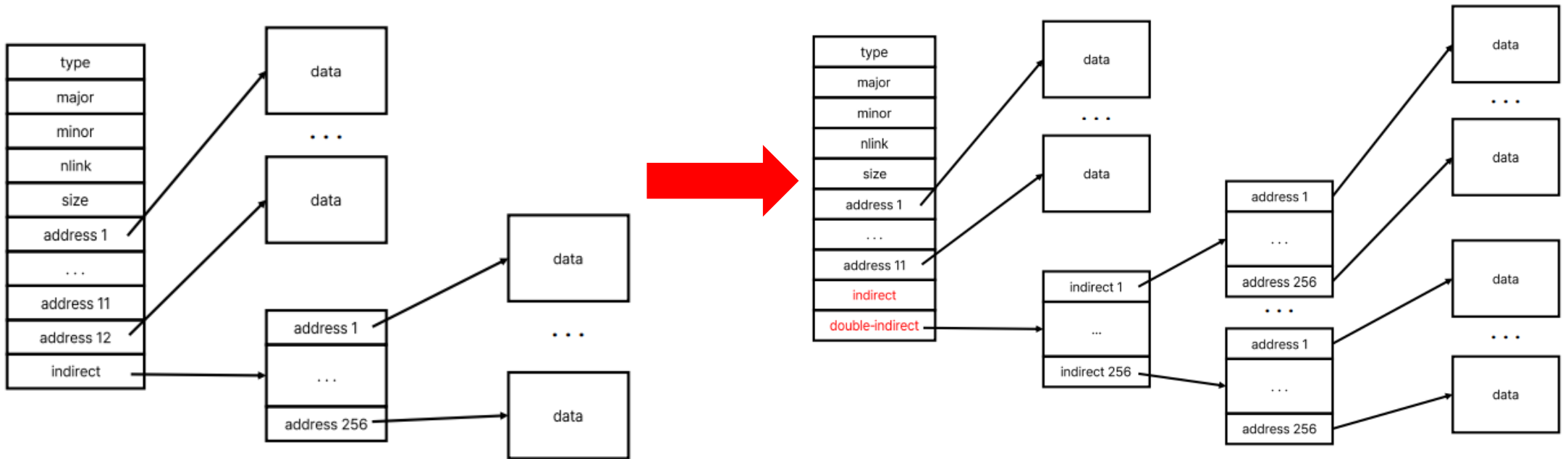
    if(r){
        memset((char*)r, 5, PGSIZE); // fill with junk
        acquire(&ref_lock);
        ref_count[(uint64)r / PGSIZE] = 1;
        release(&ref_lock);
    }
    return (void*)r;
}
```

Design – Large files

- kernel/fs.h, fs.c

기존 xv6는 파일의 최대 크기가 NDIRECT + NINDIRECT에 제한되어 있다.

이를 해결하기 위해 Doubly Indirect Block을 추가하여 MAXFILE을 확장할 수 있다.



Implementation Overview

- kernel/fs.h
 1. NDIRECT, MAXFILE 수정
 2. struct dinode 수정
 3. NIINDIRECT 추가
- kernel/file.h
 1. struct inode 수정
- kernel/sysfile.c
 1. create() 수정
- kernel/param.h
 1. FSIZE 수정
- kernel/fs.c
 1. bmap() 수정
 2. itrunc() 수정

Implementation – Large files

- kernel/fs.h
 - 1. NDIRECT, MAXFILE 수정
 - 2. struct dinode 수정
 - 3. NIINDIRECT 추가
- kernel/file.h
 - 1. struct inode 수정

과제 명세에 따라 NDIRECT를 12에서 11로 바꿔주고 FSIZE를 200000으로 바꿔줬다.

doubly-indirect blocks을 지원하기 위해 NIINDIRECT를 NINDEIRECT * NINDEIRECT로 define한다.

MAXFILE 크기 역사 NIINDIRECT를 추가로 더한 값으로 define한다.

doubly-indirect blocks도 존재하기 때문에 addr[]의 크기를 NDIRECT+1에서 NDIRECT+2로 1 증가시켜준다.

이는 dinode와 inode 둘 다 적용된다.
- kernel/sysfile.c
 - 1. create() 수정

새로운 inode를 생성 시 addr[]를 0으로 초기화 해주기 위해서 memset()을 이용해 크기가 바뀐 addr[]를 초기화 해준다.

kernel/param.h

- 1. FSIZE 수정

fs.h

```
#define NDIRECT 11
#define NINDIRECT (BSIZE / sizeof(uint))
#define NIINDIRECT (NINDIRECT * NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NIINDIRECT)

// On-disk inode structure
You, 7 hours ago | 2 authors (github-classroom[bot] and one other)
struct dinode {
    short type;           // File type
    short major;          // Major device number (T_DEVICE only)
    short minor;          // Minor device number (T_DEVICE only)
    short nlink;          // Number of links to inode in file system
    uint size;            // Size of file (bytes)
    uint addrs[NDIRECT+2]; // Data block addresses
};
```

file.h

```
struct inode {
    uint dev;             // Device number
    uint inum;            // Inode number
    int ref;              // Reference count
    struct sleeplock lock; // protects everything below here
    int valid;            // inode has been read from disk?

    short type;           // copy of disk inode
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT+2];
};
```

sysfile.c/create()

```
if((ip = ialloc(dp->dev, type)) == 0){
    iunlockput(dp);
    return 0;
}

ilock(ip);

memset(ip->addrs, 0, sizeof(ip->addrs));

ip->major = major;
ip->minor = minor;
ip->nlink = 1;
```

param.h

```
#define FSIZE 200000
```

Implementation – Large files

- kernel/fs.c

1. bmap 수정

bmap()은 inode의 n번째 data block에 해당하는 실제 disk block 번호를 return하는 함수이다.

기존 bmap은 direct block과 single indirect block을 지원하고 있다. single indirect block 방식을 참고해서 double indirect block을 구현했다.

direct block과 single indirect block을 처리한 이후에 bn을 double indirect block 기준으로 재조정 한다.

이때 2단계 index 구조로 접근한다.

i1: double indirect block의 entry 번호

i2: 해당 single indirect block 내에서의 offset

i1번째에 해당하는 single indirect block이 없으면 새 블록을 할당하고 해당 entry에 저장한 뒤 journaling을 위해 log_write()를 호출한다. 그리고 bp를 해제한다.

이제 방금 접근한 single indirect block을 다시 읽어오고 기존 single indirect block 접근과 똑같이 처리해준다.

```
static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;

    if(bn < NDIRECT){
        if((addr = ip->addrs[bn]) == 0){
            addr = balloc(ip->dev);
            if(addr == 0)
                return 0;
            ip->addrs[bn] = addr;
        }
        return addr;
    }
    bn -= NDIRECT;

    if(bn < NINDIRECT){
        // Load indirect block, allocating if ne
        if((addr = ip->addrs[NDIRECT]) == 0){
            addr = balloc(ip->dev);
            if(addr == 0)
                return 0;
            ip->addrs[NDIRECT] = addr;
        }
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[bn]) == 0){
            addr = balloc(ip->dev);
            if(addr){
                a[bn] = addr;
                log_write(bp);
            }
        }
        brelse(bp);
        return addr;
    }
}
```

```
bn -= NINDIRECT;

if (bn < NIINDIRECT) {
    if ((addr = ip->addrs[NINDIRECT + 1]) == 0) {
        addr = balloc(ip->dev);
        if (addr == 0)
            return 0;
        ip->addrs[NINDIRECT + 1] = addr;
    }

    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    uint i1 = bn / NINDIRECT;
    uint i2 = bn % NINDIRECT;

    if ((addr = a[i1]) == 0) {
        addr = balloc(ip->dev);
        if (addr == 0) {
            brelse(bp);
            return 0;
        }
        a[i1] = addr;
        log_write(bp);
    }
    brelse(bp);

    bp = bread(ip->dev, a[i1]);
    a = (uint*)bp->data;

    if ((addr = a[i2]) == 0) {
        addr = balloc(ip->dev);
        if (addr) {
            a[i2] = addr;
            log_write(bp);
        }
    }
    brelse(bp);
    return addr;
}
```

Implementation – Large files

- kernel/fs.c

- 2. itrunc 수정

이 함수는 파일을 삭제하거나 크기를 줄일 때 inode에 연결된 모든 data block을 해제한다.

기존 함수는 direct block과 indirect block을 해제하고 있어서 indirect block을 해제하는 방법을 참고해 double indirect block을 해제하는 코드를 추가했다.

기존에 indirect block을 해제할 때 bp, a 변수를 사용했는데 일관성을 위해서 bp1, a1으로 수정해줬다.

```
void
itrunc(struct inode *ip)
{
    int i, j, k;
    struct buf *bp1, *bp2;
    uint *a1, *a2;

    for(i = 0; i < NDIRECT; i++){
        if(ip->addrs[i]){
            bfree(ip->dev, ip->addrs[i]);
            ip->addrs[i] = 0;
        }
    }

    if(ip->addrs[NDIRECT]){
        bp1 = bread(ip->dev, ip->addrs[NDIRECT]);
        a1 = (uint*)bp1->data;
        for(j = 0; j < NINDIRECT; j++){
            if(a1[j])
                bfree(ip->dev, a1[j]);
        }
        brelse(bp1);
        bfree(ip->dev, ip->addrs[NDIRECT]);
        ip->addrs[NDIRECT] = 0;
    }

    if(ip->addrs[NDIRECT + 1]){
        bp1 = bread(ip->dev, ip->addrs[NDIRECT + 1]);
        a1 = (uint*)bp1->data;
        for(j = 0; j < NINDIRECT; j++){
            if(a1[j]){
                bp2 = bread(ip->dev, a1[j]);
                a2 = (uint*)bp2->data;
                for(k = 0; k < NINDIRECT; k++){
                    if(a2[k])
                        bfree(ip->dev, a2[k]);
                }
                brelse(bp2);
                bfree(ip->dev, a1[j]);
            }
        }
        brelse(bp1);
        bfree(ip->dev, ip->addrs[NDIRECT + 1]);
        ip->addrs[NDIRECT + 1] = 0;
    }

    ip->size = 0;
    iupdate(ip);
}
```

Design – Symbolic links

symbolic link는 file system에서 다른 file의 경로를 가리킨다.

문자열로 된 경로만을 저장하고 이 경로를 통해 원래의 file에 간접적으로 접근한다.

기존 file과는 다른 특수한 file이기 때문에, 기존 xv6가 file을 처리하는 방식을 변형 해 symbolic link를 지원할 수 있게 해야 한다.

Implementation Overview

- kernel/stat.h
 1. T_SYMLINK 추가
- kernel/fcntl.h
 1. O_NOFOLLOW 추가
- kernel/sysfile.c
 1. sys_open() 수정
 2. create() 수정
 3. sys_symlink() 추가
- user/user.h, usys.pl
 - syscall 만들기

Implementation – Symbolic links

- kernel/sysfile.c
 1. T_SYMLINK 추가
inode type에 symbolic link를 추가한다.
- kernel/fcntl.h
 1. O_NOFOLLOW 추가
임의로 0x0100을 O_NOFOLLOW로 정의한다.
symbolic link를 따라가지 않도록 하는 역할이다.

```
#define T_DIR      1    // Directory
#define T_FILE     2    // File
#define T_DEVICE   3    // Device
#define T_SYMLINK  4    // symbolic link
```

```
#define O_RDONLY  0x000
#define O_WRONLY  0x001
#define O_RDWR    0x002
#define O_CREATE  0x200
#define O_TRUNC   0x400
#define O_NOFOLLOW 0x0100
```

Implementation – Symbolic links

- kernel/sysfile.c

1. sys_open 수정

file open 과정에서 symbolic link를 따라가는 동작을 추가한다.

현재 inode가 T_SYMLINK이고 O_NOFOLLOW가 지정되지 않은 경우 link를 따라가는데, 이때 무한 루프 방지를 위해 depth counter를 사용해 최대 10단계까지만 따라갈 수 있도록 한다.

이후 inode를 정리하고 end_op()로 종료한다.

link 내부의 path를 readi()로 읽고 namei()를 사용해 읽어온 경로로 다시 inode를 탐색한다.

2. create() 수정

새 file을 생성할 때 해당 path에 이미 존재하는 file이 있는 경우 새로 만들지 않고 기존 inode를 그대로 반환하는데, 이때 새로 만든 type인 T_SYMLINK까지 반영한다.

```
} else {
    if((ip = namei(path)) == 0){
        end_op();
        return -1;
    }
    ilock(ip);
    if ((ip->type == T_SYMLINK) && !(omode & O_NOFOLLOW)) {
        int depth = 0;
        while(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)) {
            if (++depth >= 10) {
                // prevent infinite loop
                iunlockput(ip);
                end_op();
                return -1;
            }

            char target[MAXPATH];
            int len = readi(ip, 0, (uint64)target, 0, MAXPATH - 1);
            if(len < 0){
                iunlockput(ip);
                end_op();
                return -1;
            }
            target[len] = '\0';

            iunlockput(ip);
            ip = namei(target);
            if(ip == 0){
                end_op();
                return -1;
            }
            ilock(ip);
        }
        if(ip->type == T_DIR && omode != O_RDONLY){
            iunlockput(ip);
            end_op();
            return -1;
        }
    }
}
```

```
if(type == T_FILE && (ip->type == T_FILE || ip->type == T_DEVICE || ip->type == T_SYMLINK))
    return ip;
iunlockput(ip);
return 0;
```


Implementation – Symbolic links

- kernel/sysfile.c
 - 3. sys_symlink() 추가
symbolic link file을 생성하고 그 안에 target path 문자열을 저장한다.
- user/user.h, usys.pl
 - syscall 만들기
syscall 만드는 방법은 생략하겠다.

```
uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;

    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;

    begin_op();
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }

    if (writei(ip, 0, (uint64)target, 0, strlen(target)) != strlen(target)) {
        iunlockput(ip);
        end_op();
        return -1;
    }

    iunlockput(ip);
    end_op();
    return 0;
}
```

Results

- 테스트 파일 cowtest.c, bigfile.c, symlinktest.c 실행 결과이다.

```
xv6 kernel is booting

init: starting sh
$ cowtest
simple: ok
simple: ok
three: ok
three: ok
three: ok
file: ok
ALL COW TESTS PASSED
$ symlinktest
Start: test symlinks
test symlinks: ok
Start: test concurrent symlinks
test concurrent symlinks: ok
$
```

```
xv6 kernel is booting

init: starting sh
$ bigfile
.....
.....
.....
.....
.....
wrote 65803 blocks
bigfile done; ok
$
```

Troubleshooting

- page fault가 발생할 때 xv6가 어떻게 처리하는지 그 과정에 대한 이해가 부족했어서 그 과정을 이해하기 위한 시간을 많이 쏟았다. riscv-xv6 manual 문서를 읽으면서 store page fault는 `scause==15`임을 알았다.
- symbolic link 과제를 할 때, `panic: virtio_disk_intr status`가 발생했는데 그 원인을 알 수 없었다. 원인을 찾기 위해 여러 시도를 하다가 그 이유를 결국 알게 되었는데 바로 두 번째 과제 명세를 꼼꼼히 읽지 않아서 생긴 문제였다. doubly-indirect block을 지원하면서 `dinode`의 struct 중 `addrs[]`를 수정했지만, `inode`의 `addrs[]`는 수정하지 않았다. `bigfile.c` test는 `inode`의 struct를 수정하지 않아도 문제가 없었지만, `symlinktest.c`에서 문제가 생긴 것이다. 지금 세 번째 과제 구현 자체는 잘 했는데 두 번째 과제 구현의 실수로 생긴 문제였어서 더더욱 문제를 찾기 어려웠다.